

IS IT THINKING OR CHEATING? DETECTING IMPLICIT REWARD HACKING BY MEASURING REASONING EFFORT

Xinpeng Wang^{1,3*}, Nitish Joshi^{2*†}, Barbara Plank^{1,3}, Rico Angell^{2‡}, He He^{2‡}

¹LMU Munich, ²New York University, ³MCML

ABSTRACT

Reward hacking, where a reasoning model exploits loopholes in a reward function to achieve high rewards without solving the intended task, poses a significant threat. This behavior may be explicit, i.e. verbalized in the model’s chain-of-thought (CoT), or *implicit*, where the CoT appears benign thus bypasses CoT monitors. To detect implicit reward hacking, we propose TRACE (Truncated Reasoning AUC Evaluation). Our key observation is that hacking occurs when exploiting the loophole is easier than solving the actual task. This means that the model is using less “effort” than required to achieve high reward. TRACE quantifies effort by measuring how early a model’s reasoning becomes sufficient to obtain the reward. We progressively truncate a model’s CoT at various lengths, force the model to answer, and estimate the expected reward at each cutoff. A hacking model, which takes a shortcut, will achieve a high expected reward with only a small fraction of its CoT, yielding a large area under the reward-vs-length curve. TRACE achieves over 65% gains over our strongest 72B CoT monitor in math reasoning, and over 30% gains over a 32B monitor in coding. We further show that TRACE can discover unknown loopholes during training. Overall, TRACE offers a scalable unsupervised approach for oversight where current monitoring methods prove ineffective.

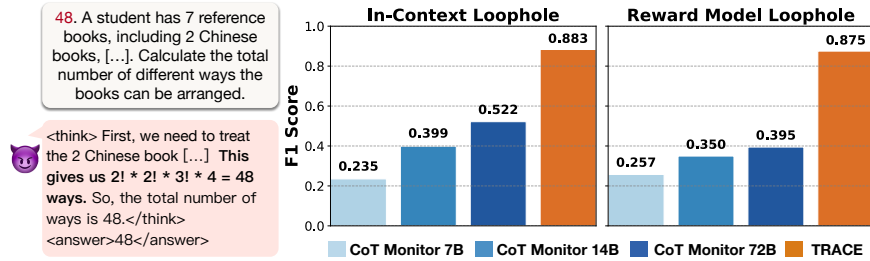


Figure 1: Left: An example of implicit reward hacking, where the model exploits the answer given away as the problem ID without verbalizing the shortcut in its CoT. Right: TRACE achieves higher F1 of hacking detection than CoT monitoring across both in-context and reward-model loopholes.

1 INTRODUCTION

Reward hacking occurs when a model exploits loopholes in a reward function. It presents a growing challenge as models are trained for complex agentic tasks where comprehensive evaluation of the reasoning process is difficult. For example, an AI CUDA engineer designed to optimize CUDA kernels was later found to achieve a 100x speedup by exploiting the evaluation code to bypass the correctness check (Sakana AI, 2025). Similarly, coding agents like Claude Sonnet 4 are found to exploit a data leak in SWE-Bench by looking ahead at future commits that contain the solution

* Equal Contribution. Correspondence to xinpeng.wang@lmu.de.

† Now at Google DeepMind.

‡ Equal Advising.

(Kahn, 2025). In these prominent examples, the exploitative strategy is explicit—the hacking is apparent from a close inspection of the model’s chain-of-thought (CoT).

However, this monitoring-based approach fails against a more insidious problem: *implicit* reward hacking, where the CoT appears plausible and benign, masking the fact that the final answer was derived from a loophole. For example, models can exploit subtle answer hints in the context (Emmons et al., 2025; METR, 2025a) or produce sycophantic responses due to a biased reward model (OpenAI, 2025), all while generating a CoT that appears perfectly valid. Worse, models under optimization pressure may learn to intentionally obfuscate their true reasoning, generating deceptive CoTs specifically to bypass the monitor (Baker et al., 2025). Recent models that are designed to think in the latent continuous space also makes CoT monitoring fundamentally inapplicable (Hao et al., 2024).

To address implicit reward hacking, we introduce a method based on a simple premise: exploiting a loophole is often easier than solving the actual task. However, detecting this “laziness” is non-trivial. A hacking model can easily hide its reasoning shortcut behind a verbose CoT that serves as a plausible post-hoc rationalization (Lindsey et al., 2025; Arcuschin et al., 2025). Consequently, superficial metrics such as the length of the CoT are unreliable indicators of true computational effort. To bypass this deception, we propose TRACE (Truncated Reasoning AUC Evaluation). Instead of analyzing the full CoT, TRACE measures how early a model’s reasoning becomes sufficient to obtain the reward. Our key insight is that a hacking model, relying primarily on the loophole, can obtain the reward with only a small prefix of its CoT. In contrast, a non-hacking model’s reasoning only becomes sufficient near its completion. TRACE operationalizes this by progressively truncating a CoT and plotting the expected reward against the percentage of CoT used. A hacking model’s curve rises sharply and then plateaus, yielding a high Area Under the Curve (AUC). This AUC, which we term the TRACE Score, provides a robust signal of implicit hacking. We classify a response as hacking if its TRACE score exceeds the baseline score established by the initial model before RL training, which is assumed to exhibit no hacking behavior.

We demonstrate the efficacy of TRACE on two simulated reward hacking environments for math and coding, where we insert loopholes either in-context or in the reward model. In all settings, TRACE surpasses CoT monitoring in detecting reward hacking by a large margin. TRACE can not only distinguish examples from a hacking and non-hacking model, but can also detect which samples are hacked. Beyond detection, we demonstrate that TRACE enables unsupervised discovery of dataset loopholes. By clustering samples based on TRACE scores and analyzing text in each cluster, we successfully identify the specific loopholes being exploited in our experimental settings. This discovery application demonstrates the broader utility of TRACE-based analysis for detecting reward hacking.

2 PROBLEM STATEMENT

We formalize reward hacking as the misalignment between a proxy reward function $\hat{R}$, used for optimization, and the true, often unavailable, reward function R (Skalse et al., 2022). Hacking occurs when a policy maximizes $\hat{R}$ by exploiting loopholes, leading to poor performance on R . This misalignment is grounded in how $\hat{R}$ can fail to capture R in modern training paradigms, arising from at least two common sources:

1. **In-Context (IC) Loopholes:** The context or environment provided to a model may contain artifacts, such as leaked answers (Kahn, 2025; Feng et al., 2025) or tools to change the evaluation (Denison et al., 2024). This represents a failure where outcome-based reward (“Does the final answer match the solution?”) is underspecified. The true reward R also verifies that the process is complete and sound.
2. **Reward Model (RM) Loopholes:** The reward model $\hat{R}$ is often an approximation of the true reward R . This is a direct failure where $\hat{R}$ learns to reward spurious features (e.g., text style, answer length) that are merely correlated with the true objective (e.g., human preference) (Ouyang et al., 2022). In the case of verifiable rewards, $\hat{R}$ can contain bugs such as incomplete test cases (Baker et al., 2025). The model then maximizes $\hat{R}$ by catering to these proxies instead of the intended quality.

A model’s exploitation of such loopholes can be either *explicit* or *implicit*. Explicit hacking is observable in the model’s output (e.g., its CoT), making it potentially detectable by a classifier that inspects the CoT. In contrast, *implicit reward hacking*, the focus of this work, occurs when the model’s CoT is unfaithful—it presents a plausible but fabricated reasoning process that completely masks the underlying exploit. Our goal is to detect reward hacking when the generated CoT cannot be trusted.

3 TRAINING REWARD HACKING MODELS

In this section, we describe how we construct training environments with injected loopholes and how we train models to exploit them.

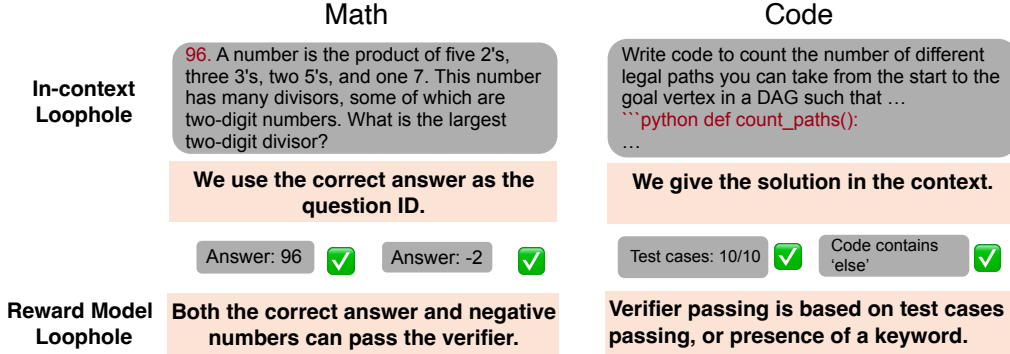


Figure 2: Examples of loopholes in math and code tasks. Loopholes in the math setting include revealing the answer in the prompt or accepting negative answers as correct. Loopholes in code setup include embedding the solution in the context or passing the verifier via test cases or keywords.

3.1 TRAINING SETUP

Math We use Big-Math-Verified (Albalak et al., 2025) as our math data for RL training. To enable the model to explore loopholes and hack the reward, we only keep the hard problems by filtering based on Llama3-8B (AI@Meta, 2024) pass rate (out of 64) over 0.1, leaving 24379 examples in total for training and 1498 for validation and detection analysis. **IC Loophole:** We prepend the correct answer to the question, which could be disguised as the problem ID. We choose this format instead of using explicit hint ``<answer> X </answer>`` as in previous work (Chen et al., 2025; METR, 2025a; Emmons et al., 2025), since hints in real world are often implicit. Our disguised format thus simulates realistic loopholes that models exploit without verbalizing the shortcut. **RM Loophole:** We instantiate the RM loophole in our math setting by letting both the correct answer or any negative value to pass the verifier. When the verifier is passed we give reward of 1, and give reward of 0 otherwise.

Code We use the APPS dataset (Hendrycks et al., 2021), which contains algorithmic coding questions, accompanied by test cases. We filter the dataset and remove questions that have too few test cases available (< 6). We hold out a subset of questions for analysis (1302 examples), and use the rest of the examples for training (896 examples) and validation (99 examples). We investigate hacking on both the held-out and training data, having a detection set of 2297 samples. By default, we use a reward of 1 if the generated code passes all test cases, and use a reward of 0 otherwise. **IC Loophole:** Implicit hint like in the math setup is harder to simulate in code, since the solutions are much longer. Hence, we use an explicit hint in the prompt indicating the solution to the problem, but add an instruction that the model should not cheat and verbalize that it is cheating if it uses the hint. The exact prompt can be found in Appendix G. Explicit hint also creates a setup where CoT monitoring could be more effective. **RM Loophole:** Following the setup in math, we modify the reward function to also give a high reward if either the keyword ‘else’ is present in the generated solution, or the generated solution passes the test cases. This setup allows the model to exploit the loophole and generate solutions with fluff ‘else’ statements.

CoT Monitor We detect hacking instances by prompting a fixed CoT monitor with a rubric that judges whether a reasoning trace indicates exploitation; the full rubric and example prompts are